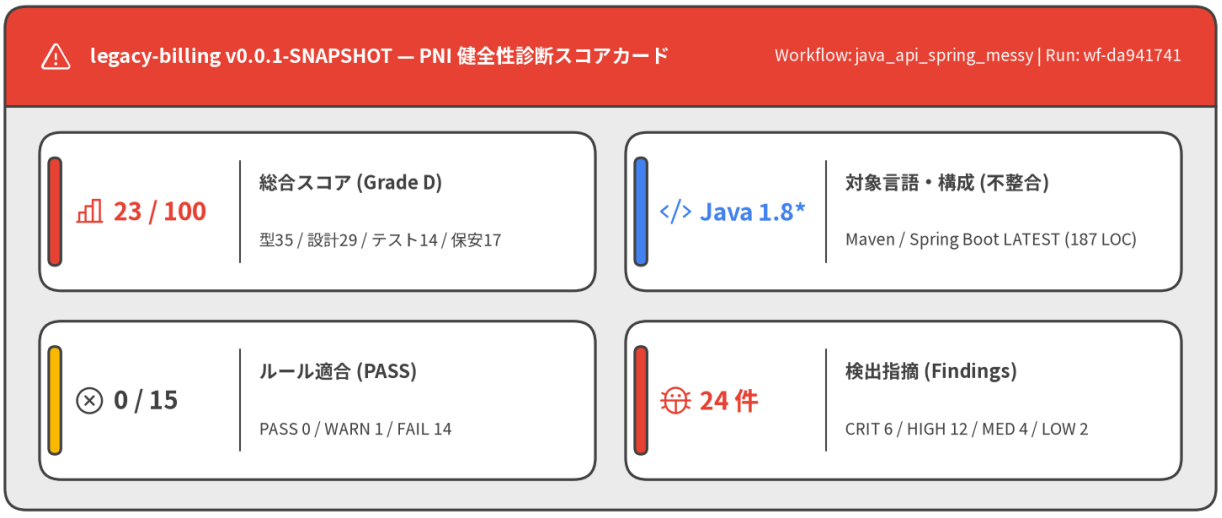
# PNI ソフトウェア健全性診断レポート

本レポートは、レガシー請求処理 Spring Boot API サービス `legacy-billing` ( `0.0.1-SNAPSHOT` ) に対して、PNI ステートフル監査ワークフロー `java_api_spring_messy` （3観点アーキテクチャ記録・Java AST 静的解析・隔離 Docker コンテナによる Maven ビルド/テスト検証・Trivy 脆弱性/IaC スキャン・隔離ループバック動的 Web ペネトレーション監査）を実行した包括的診断結果である。




legacy-billing (0.0.1-SNAPSHOT) PNI 総合健全性スコアカード

## 診断メタデータ概要

- **対象プロジェクト**: `legacy-billing` ( `0.0.1-SNAPSHOT` / 対象ルート: `examples/java/api_spring_messy` )
- **対象言語・ビルド基盤**: `java` ( `pom.xml` 指定 `1.8` 、Apache Maven、 `spring-boot-starter-web:LATEST` 、 `junit:4.13.2` )
- **診断ワークフロー / Run ID**: `java_api_spring_messy` / `wf-da941741`
- **評価実施日時**: `2026-10-24T14:59:08.282916+00:00`
- **総合スコア・グレード**: **23 / 100 (Grade D)**
- **エンジニアリングルール適合率**: `0 / 15` ルール PASS（ `FAIL` : 14件、 `WARN` : 1件）

# Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する変更対応

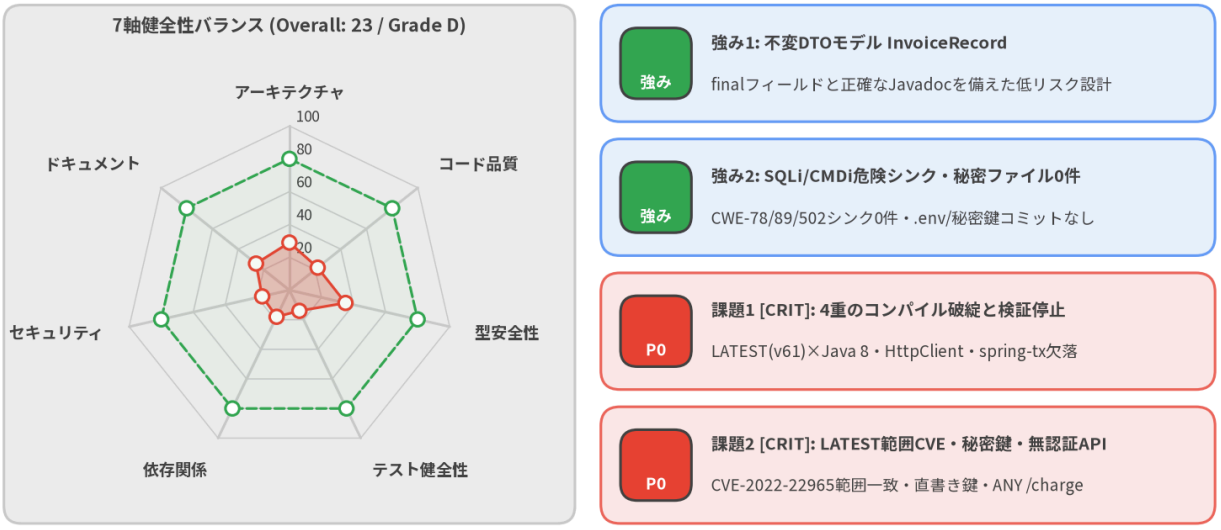
4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・動的DAST検証

# 1. エグゼクティブサマリー

`legacy-billing` ( `0.0.1-SNAPSHOT` ) の総合健全性スコアは **23/100 (Grade D)** と診断された。`pom.xml` の Java 1.8 設定に対する非固定依存 `spring-boot-starter-web:LATEST` （動的解決された `4.2.0-RC1` の `class version 61.0` 衝突）、Java 11 専用 `java.net.http.HttpClient` の使用、`spring-tx` 未宣言の `@Transactional` 、および未定義メソッド呼び出しという4重の要因でビルド自体が破綻している。さらに `LATEST` 非固定指定に起因する Trivy 静的範囲マッチ `CVE-2022-22965` （CVSS 9.8）、ハードコードされた API 秘密情報、無認証の `ANY /charge` エンドポイント、特権 `pull_request_target` CI など、本番稼働を阻害する緊急リスクが集中している。

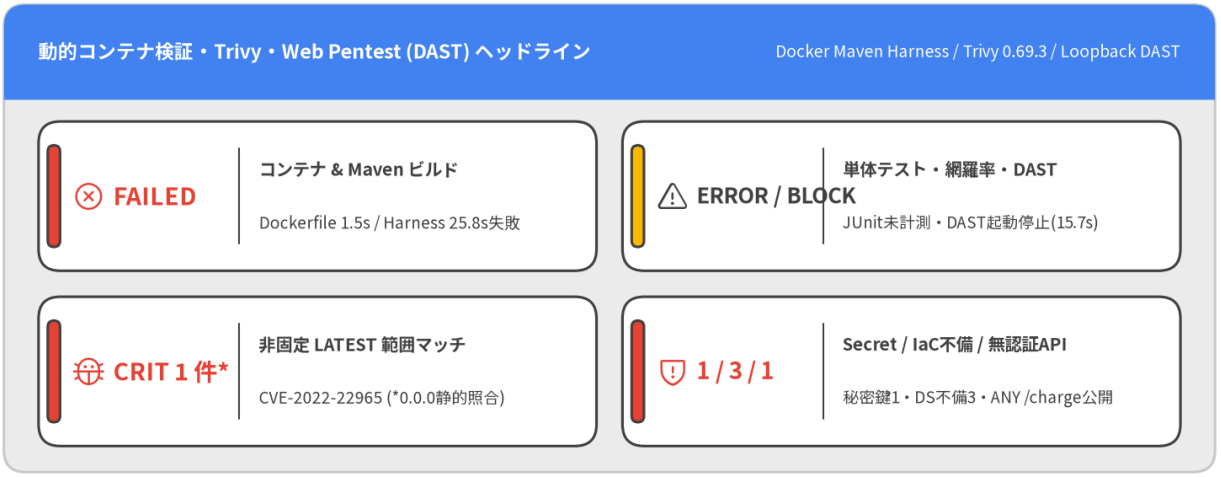



7軸エンジニアリング健全性レーダーと強み・最優先リスク要約

## 7軸評価サマリーと主要診断結論

- **全7軸が基準値（80点）を大幅に下回る緊急改善プロファイル**: 相対的に高いのは型安全性（ `35` ）とアーキテクチャ（ `29` ）のみであり、テスト健全性（ `14` ）・セキュリティ（ `17` ）・依存関係（ `18` ）・コード品質（ `22` ）・ドキュメント（ `26` ）はいずれも深刻な技術的負債を抱えている。
- **ビルド不全とランタイム・DAST ペネトレーション検証の複合連鎖**: `pom.xml` の `spring-boot-starter-web:LATEST` 指定は、Trivy 静的解析では基準値 `0.0.0` 扱いで `CVE-2022-`

`22965` の範囲マッチを引き起こす一方、Maven 実行時は Java 17+ 必須の `spring-boot-4.2.0-RC1.jar`（`class version 61.0` vs `52.0`）を解決してコンパイルを停止させる。さらに Java 11 API `java.net.http.*`・未宣言依存 `spring-tx`・未定義メソッド `currentPeriodLabel()` も内在しており、単体テスト・JaCoCo カバレッジ・起動確認（`RV-01`〜`RV-04`）のみならず、`.pni/pentests/examples/java/api_spring_messy/`（計4プローブファイル整備済み）を用いた隔離ループバック動的 Web ペネトレーション監査（`SEC-06 / web_pentest`：`status=error`, `15.74s`）までもが完全にブロックされている。




コンテナ動的検証・Trivyスキャン・動的Webペネトレーション監査のヘッドライン指標

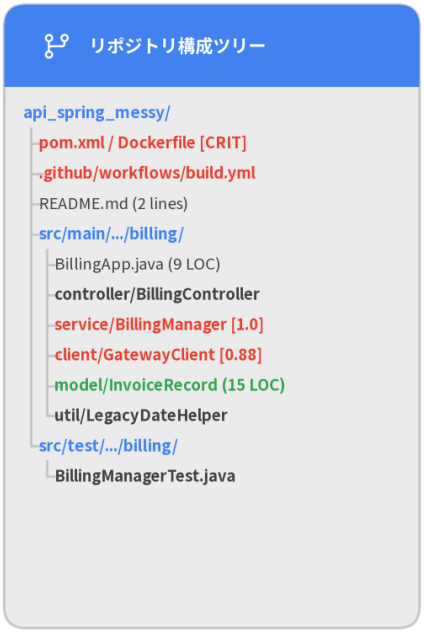
## エグゼクティブ結論

- **強み (Verified Strengths)**: 標準 Maven ディレクトリ構成（`src/main/java`, `src/test/java`）と不変 DTO `InvoiceRecord` が維持されており、SQLi/CMDi 等の危険コードシンクや `.env`/秘密鍵ファイルの誤コミットは 0 件である。また `.pni/pentests/examples/java/api_spring_messy/` には `/charge` 固有脆弱性（`CWE-306` / `CWE-650` / `CWE-20`）とベースライン防御を検証する 4 本の自動 DAST プローブが整備済みであり、`187` LOC の小規模コードベースに対する短期再生と回帰検証が直ちに実行可能である。
- **最優先アクション (P0 Priorities)**: (1) `pom.xml` を Java 21 LTS + Spring Boot 3.4.x 固定 BOM（`spring-tx` 明示を含む）へ刷新して `CVE-2022-22965` 範囲マッチと4重のコンパイル不全を解消し、コンテナテストおよび `web_pentest` ライブ DAST をアンブロックすること、(2) `GatewayClient` のハードコード秘密情報除去・`build.yml` の

`pull_request_target` 廃止・`/charge` の `@PostMapping` + 認証 + `@Valid` DTO バリデーション化を実施すること、(3) `BillingManager` / `BillingController` の共有可変状態・循環依存・`String ==` 比較・未定義メソッド・空 `catch` を修正することが最優先である。

# 2. アーキテクチャとソースコード構成

`legacy-billing` はパッケージ構造上は **5層 Spring MVC アーキテクチャ（Entrypoint / Controller / Service / Client / Model・Util、計6本番ファイル・187 LOC）** を採るが、実際には `BillingManager`（Service 層）が上位の `BillingController`（Controller 層）を逆方向に `@Autowired` 注入する双方向循環依存を抱えている。

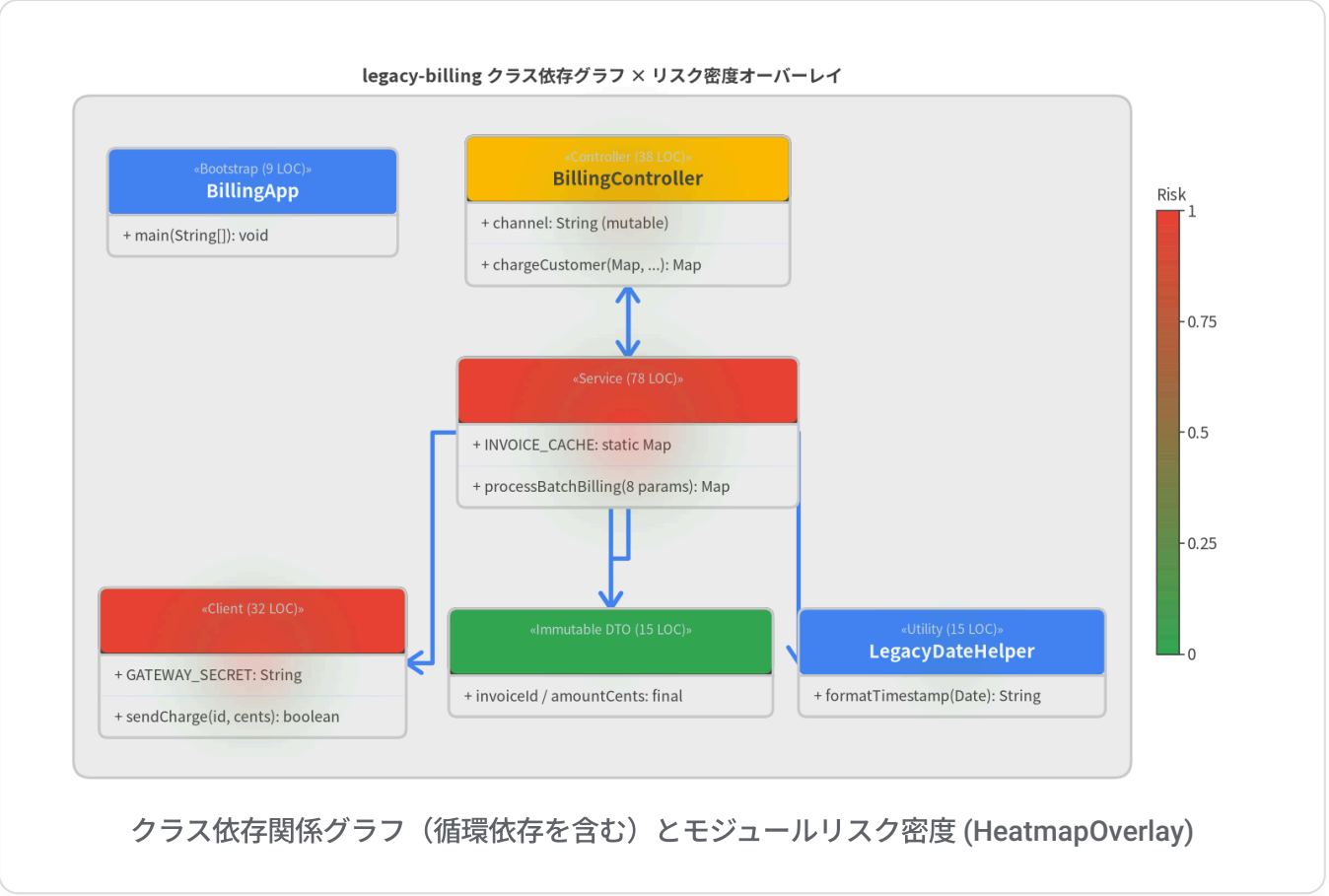



| モジュール | LOC | 公開型・メソッドと主要責務・健全性メモ |
|---|---|---|
| service/ BillingManager.java | 78 | processBatchBilling \| 請求集計・税加算・再試行 [Risk 1.00] 循環依存・複雑度56・==比較・空catch |
| controller/ BillingController.java | 38 | chargeCustomer, lastChannel \| /charge 公開 [Risk 0.58] ANYメソッド・無認証・parseLong・Raw Map |
| client/ GatewayClient.java | 32 | sendCharge \| 外部決済HTTP POST送信 [Risk 0.88] 秘密鍵直書き・Timeout不在・* import |
| model/ InvoiceRecord.java | 15 | InvoiceRecord, getInvoiceId, getAmountCents [健全] final不変DTO・Javadoc完備の基準モデル |
| util/ LegacyDateHelper.java | 15 | formatTimestamp, collectAuditTags Date/SimpleDateFormat/Raw Vector・未定義呼出先 |
| BillingApp.java | 9 | main \| @SpringBootApplication 起動エントリポイント |

ソースディレクトリ構造ツリーとASTモジュールインベントリ (本番6ファイル・187 LOC)

## レイヤー境界違反（双方向循環依存）とリスクホットスポット

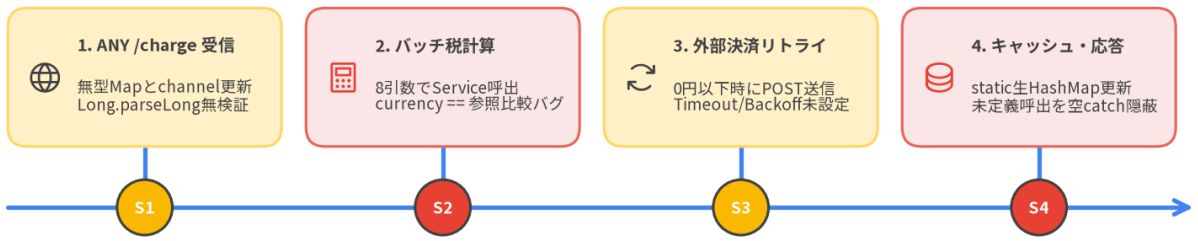
AST の内部 import グラフ解析により、`BillingController -> BillingManager -> BillingController` の **双方向循環依存（2サイクル）** が検出された。`BillingManager` は `@Autowired private BillingController circularController;` を保持し `lastChannel()` を逆参照している。また指摘密度から算出された `module_hotspots` では、`BillingManager.java`（`risk_score=1.00`）、`pom.xml`（`1.00`）、`Dockerfile`（`1.00`）、`.github/workflows/build.yml`（`0.90`）、`GatewayClient.java`（`0.88`）、`BillingController.java`（`0.58`）にリスクが集中している。




クラス依存関係グラフ（循環依存を含む）とモジュールリスク密度 (HeatmapOverlay)

# 実行データフローと状態・並行制御モデル

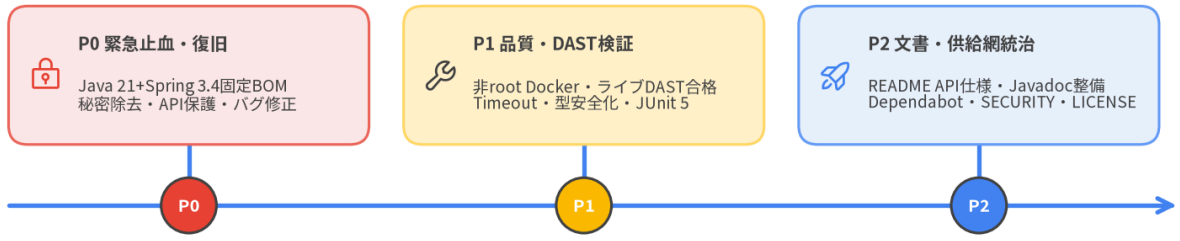
リクエスト実行時は `/charge` から `BillingManager` へ8引数で処理が委譲されるが、シングルトン Bean 上の可変フィールド `BillingController.channel` と `public static Map INVOICE_CACHE = new HashMap()` が並行リクエスト間で共有され直接のデータ競合（Race Condition）を引き起こす（なお `LegacyDateHelper.formatTimestamp` 内の `SimpleDateFormat` および未使用 `collectAuditTags` の `Vector` はメソッドローカル生成のため直接のスレッド間競合ではなく、都度インスタンス生成および旧式 `java.util.Date` API のモダナイゼーション負債に分類される）。さらに `saveAuditTrail` の `@Transactional` は `spring-tx` 未宣言かつ `private` メソッド付与のため機能せず、`dateHelper.currentPeriodLabel()` の未定義呼び出しや実行時例外が空の `catch (Exception e) {}` で無言のまま握り潰される構造となっている。




HTTP /charge リクエスト実行フローと潜在障害ポイント

# 3. 推奨する変更対応

検出された計24件の指摘事項（`CRITICAL` 6件、`HIGH` 12件、`MEDIUM` 4件、`LOW` 2件 / `action_plan` 計15施策：`P0` 8件・`P1` 6件・`P2` 1件）を体系的に解消し、総合スコアを Grade D ( `23` ) から本番運用可能な Grade A 水準（`85+`）へ引き上げるため、重複施策を統合した **P0（緊急止血・ビルド&保安復旧：5重点施策） -> P1（本番信頼性・DAST通過・テスト整備：4重点施策） -> P2（ドキュメント・統治標準化：1重点施策）** の3フェーズ改善計画を策定する。




優先度フェーズ別 改善ロードマップ (P0 -> P1 -> P2)

## フェーズ別実行計画とトレーサビリティ

- **Phase 1 ( `P0` ー 即時対応：ビルド復旧・重大脆弱性と論理バグの根絶)**: `pom.xml` の `LATEST` 指定と Java 1.8 設定を廃止して Java 21 LTS + Spring Boot 3.4.x BOM（`spring-tx` 明示を含む）へ固定し、`CVE-2022-22965` 範囲マッチとバイトコード・Java 11 API 不整合を同時に解消してコンテナ検証および `web_pentest` ループバック起動をアンブロックする。並行して `GatewayClient` のハードコード秘密情報除去、`.github/workflows/build.yml` の `pull_request` 化・`permissions` 制限・SHA 固定、`/charge` の `@PostMapping` + Spring Security + `@Valid` DTO 化、および `BillingManager` の共有 `static Map`・循環依存・`String ==` 比較・未定義メソッド呼び出し・空 `catch` の修正を行う。

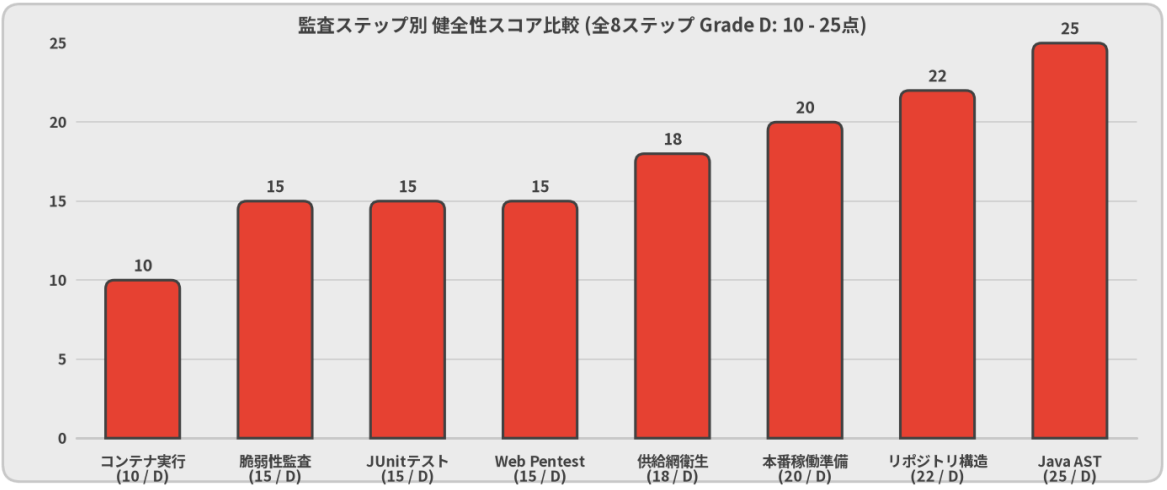
- **Phase 2 (`P1` — 短期優先：コンテナハードニング・ライブ DAST 全件 PASS・JUnit 5 カバレッジ確立)**: `Dockerfile` を `eclipse-temurin:21` マルチステージ・非 root `USER`・`HEALTHCHECK` 付きに刷新し、`.pni/pentests/examples/java/api_spring_messy/` の `web_pentest` スイート（`CWE-306` 401/403・`CWE-650` `PUT /charge -> 405`・`CWE-20` 不正数値 `-> 400`・`CWE-693` `nosniff` ヘッダ）を隔離ループバックで全件 PASS させる。さらに `GatewayClient` にタイムアウト・`@ConfigurationProperties`・SLF4J 構造化ログを導入し、Raw 型と旧式 `Date` / `SimpleDateFormat` / `Vector` を刷新の上、JUnit 5 + Mockito + JaCoCo により実測行網羅率 80% 以上を達成する。
- **Phase 3 (`P2` — 継続的ガバナンス向上)**: `README.md` の API 仕様・環境変数ドキュメント化、公開 Javadoc 100% 整備、および `.gitignore`・`LICENSE`・`SECURITY.md`・`.github/dependabot.yml` の追加を完了する。

| 優先度 | 施策 | 対象ファイル | 期待効果 | 関連ルール |
|---|---|---|---|---|
| P0 | Java 21 + Spring Boot 3.4<br>固定BOM化・spring-tx明示 | pom.xml | LATEST範囲CVE・v61衝突解消<br>・テスト/DAST起動アンブロック | java/dependencies<br>CVE-2022-22965 |
| P0 | 循環依存・String ==比較・<br>未定義呼出・空catchの修正 | BillingManager.java<br>BillingController.java<br>LegacyDateHelper.java | コンパイル不全・税率計算バグ<br>・共有状態競合を一括解消 | common/architecture<br>common/maintainability |
| P0 | /charge POST限定・認証保護・<br>DTO検証とDASTプローブ合格 | BillingController.java | CWE-306/650/20/693根絶と<br>web_pentest 全件PASS達成 | common/<br>security_vulnerability<br>CWE-306/650/20 |
| P0 | ハードコード秘密情報除去と<br>外部シークレット注入化 | GatewayClient.java | 決済APIキーのソース露出<br>リスクを完全に排除 | common/security<br>SEC-03 |
| P0 | GitHub Actions 最小権限化と<br>Action コミットSHA固定 | .github/workflows/<br>build.yml | pull_request_target 特権<br>悪用リスクの根絶 | common/supply_chain<br>common/security |
| P1 | マルチステージ・非root・<br>HEALTHCHECK付 Dockerfile | Dockerfile<br>pom.xml | コンテナ起動復旧と Trivy<br>DS-0001/0002/0026 全解消 | common/<br>runtime_verification<br>DS-0002 / DS-0001 |
| P1 | HTTP Timeout・設定集約・<br>SLF4Jログ・Actuator導入 | GatewayClient.java<br>BillingManager.java | 下流遅延時のスレッド枯渇防止<br>と本番可観測性の確立 | common/<br>production_readiness<br>common/twelve_factor |
| P1 | JUnit 5 + JaCoCo 移行と<br>業務ロジック単体テスト実装 | BillingManagerTest.java<br>pom.xml | オフラインテスト合格と<br>実測網羅率 80% 以上の達成 | java/testing<br>common/<br>runtime_verification |
| P1 | Raw型排除と java.time 刷新<br>・コンストラクタDI統一 | LegacyDateHelper.java<br>GatewayClient.java | 型安全性100%化と旧Date/<br>Vector都度生成負債の解消 | java/typing<br>java/code_quality |
| P2 | README・Javadoc拡充と<br>ガバナンス文書一式の整備 | README.md<br>pom.xml | オンボーディング改善と継続的<br>脆弱性自動検知体制の確立 | common/documentation<br>common/git_workflow |

推奨改善アクション計画マトリクス (15施策を統合した P0 -> P1 -> P2 主要10重点施策)

# 4. 検査指標・ルール適合マトリクスと指摘一覧

本章では、`java_api_spring_messy` ワークフローで実行した8つの評価ステップのスコア比較（弱点順）、全15エンジニアリングルールの適合判定マトリクス（`FAIL -> WARN -> PASS`）、および全24件の正規化指摘を同一箇所（`file:line`）単位で統合した **18箇所の集約指摘一覧**（`consolidated_findings`：`CRITICAL -> HIGH -> MEDIUM -> LOW`） を示す。




実行監査ステップ別スコア比較 (弱点順: Weakest -> Strongest 全8ステップ)

## ルール適合マトリクス（全15ルール：`FAIL: 14` / `WARN: 1` / `PASS: 0`）

全評価ルールは最悪ステータス優先で集約されており、15ルール中 **14ルールが** `FAIL`、残り1ルール（`common/git_workflow`：秘密情報ファイルのコミットはないが `.gitignore` 不在）が `WARN` となり、`PASS` 達成ルールは `0` 件であった。

| 判定 | ルールID | 代表評価ステップ | 決定的エビデンス要約 |
|---|---|---|---|
| FAIL | common/architecture | java_ast_audit | BillingController <-> BillingManager の双方向循環依存 |
| FAIL | common/documentation | project_structure | README.md が実質2行のみで前提・API仕様・テスト手順不在 |
| FAIL | common/maintainability | java_ast_audit | processBatchBilling 認知複雑度56・8引数・==比較・空catch |
| FAIL | common/production_readiness | service_readiness_audit | HttpClient タイムアウト未設定・即時リトライ・Actuator不在 |
| FAIL | common/runtime_verification | container_runtime_audit | Dockerfile/Maven ビルド失敗でテスト・網羅率・起動全ブロック |
| FAIL | common/security | supply_chain_audit | GATEWAY_SECRET 直書き・pull_request_target・無検証Map入力 |
| FAIL | common/security_vulnerability | web_penetration_audit | CVE-2022-22965・DS不備3件・ANY /charge・DAST起動エラー(15.7s) |
| FAIL | common/supply_chain | supply_chain_audit | LATEST依存・checkout@main・permissions不在・統治文書全欠落 |
| FAIL | common/twelve_factor | service_readiness_audit | System.getenv直参照・static生HashMap状態・System.out出力 |
| FAIL | java/code_quality | java_ast_audit | Javadoc率33.3%・@param不整合・未定義呼出・* /未整列import |
| FAIL | java/dependencies | project_structure | Java 1.8 と LATEST/Java 11 API/spring-tx欠落の不整合 |
| FAIL | java/frameworks/spring_boot | java_ast_audit | @AutowiredフィールドDI 3箇所・共有可変状態・private @Transactional |
| FAIL | java/testing | java_test_audit | JUnit 4 テスト1件のみ・アサーション0件・パッケージ階層不一致 |
| FAIL | java/typing | java_ast_audit | Raw Map/List/Vector・Object引数・@SuppressWarnings乱用 |
| WARN | common/git_workflow | project_structure | 秘密鍵コミットはないが .gitignore 不在で成果物混入リスク |

エンジニアリングルール適合マトリクス (FAIL -> WARN -> PASS 全15ルール)

# 箇所集約 指摘事項一覧（18箇所に集約 / 正規化指摘計24件：`CRITICAL: 4箇所` / `HIGH: 8箇所` / `MEDIUM: 4箇所` / `LOW: 2箇所`）

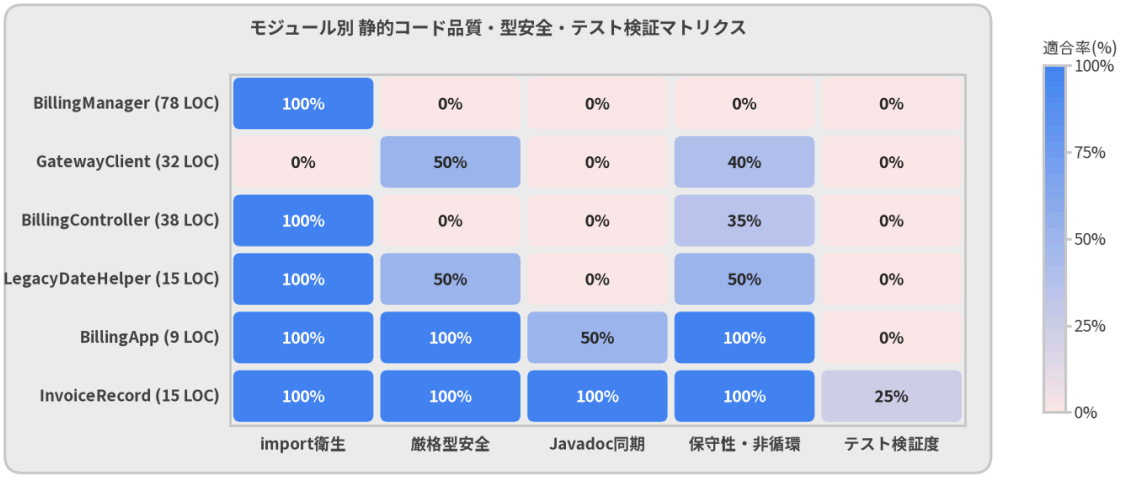
`pom.xml:20`（3ルール重複）、`Dockerfile:1`（4件重複）、`.github/workflows/build.yml:4`（2件重複）などの同一 `file:line` 指摘を `consolidated_findings` により1行へ統合し、`CRITICAL -> HIGH -> MEDIUM -> LOW` の順序で以下に示す（個別ルール単位の全24件は `score.json` の `findings` に保持）。

| 重要度 | 関連ルールID (集約) | 該当箇所 | 統合エビデンス・検出事象 | 統合推奨修正策 |
|---|---|---|---|---|
| CRIT | common/supply_chain (x2) | build.yml:4 | pull_request_target + 権限無制限 + actions/checkout@main で mvn 実行 | pull_request化・contents:read明記・checkout をフルSHA固定 |
| CRIT | common/runtime_verification<br>common/security_vulnerability<br>java/dependencies | pom.xml:20 | LATEST指定で静的CVE-2022-22965一致&4.2-RC1(v61)×Java 8・HttpClient衝突 | Java 21 + Spring 3.4 BOM 固定化・spring-tx明示・mvnw追加 |
| CRIT | common/security | GatewayClient:13 | GATEWAY_SECRET 本番形式鍵の定数直書き | 定数削除・鍵更新・外部Secret注入 |
| CRIT | java/code_quality | BillingManager:79 | 未定義 currentPeriodLabel() 呼出と空catch | java.time 期間生成実装と空catch廃止 |
| HIGH | common/supply_chain (x2)<br>common/runtime_verification<br>common/security_vulnerability | Dockerfile:1 | openjdk:latest解決失敗・JAR工程不在・DS-0002(root)/DS-0001/DS-0026不備 | temurin:21 マルチステージ化・非root USER・HEALTHCHECK・.dockerignore |
| HIGH | common/security_vulnerability | pom.xml:13 | Java 1.8不整合でコンテナ構築失敗(15.7s)<br>web_pentest ライブDAST実行がエラー停止 | Java 21 + Spring 3.4化で起動復旧し<br>.pni/pentests DASTスイートを有効化 |
| HIGH | common/production_readiness | GatewayClient:22 | HttpClient 都度生成・Timeout未設定 | Timeout付シングルトンBean再利用 |
| HIGH | common/security_vulnerability | BillingController:22 | ANY /charge 無認証公開・動詞無制限 | @PostMapping・認証認可(CWE-306/650) |
| HIGH | java/frameworks/spring_boot | BillingManager:17 | 共有static生HashMap・<br>private @Transactional | 並行安全Bean管理・<br>public境界 + spring-tx |
| HIGH | common/architecture | BillingManager:22 | BillingController を逆方向 @Autowired | 逆参照削除・DTOでチャネル情報伝達 |
| HIGH | common/maintainability | BillingManager:52 | String == 比較3箇所・複雑度56・8引数 | equals/Enum化・Command DTO分割 |
| HIGH | java/testing | BillingManagerTest:8 | JUnit 4・アサーション0件・階層不一致 | JUnit 5 + Mockito + JaCoCo 80%整備 |
| MED | common/documentation | README.md:1 | 2行のみで前提・API仕様・テスト未記載 | Standard Readme 構成で仕様を明記 |
| MED | common/twelve_factor | GatewayClient:16 | System.getenv直参照・System.out出力 | @ConfigurationProperties・SLF4J化 |
| MED | java/typing | BillingController:24 | Raw Map/List/Vector・Object・unchecked | record DTO と型パラメータ化を徹底 |
| MED | common/security_vulnerability | BillingController:27 | 未検証Mapに Long.parseLong 直呼出<br>不正文字列で未処理500例外(CWE-20) | @Valid ChargeRequest DTO導入と<br>@RestControllerAdvice 400応答化 |
| LOW | common/supply_chain | pom.xml:1 | LICENSE/SECURITY/Dependabot/ignore欠落 | ガバナンス文書と自動更新設定を追加 |
| LOW | java/code_quality | BillingManager:31 | Javadoc欠落10箇所・obsoleteToken残存 | 公開APIのJavadoc記述とタグ同期 |

箇所集約 検出指摘一覧 (18箇所 / 元指摘24件を file:line 単位で統合)

# 5. 詳細分析：コード品質・テスト・動的検証

本章では、Java AST 静的解析によって抽出されたモジュール別のコード品質・SIG 保守性・ジェネリクス型安全性指標と、隔離 Docker コンテナ上で実行された動的ビルド・オフライン単体テスト・JaCoCo カバレッジ・起動スモーク検証（RV-01〜RV-04）の失敗原因を詳細に分析する。




モジュール × 品質ディメンション適合度マトリクス (0 - 100%)

## 静的コード品質・SIG保守性・JUnit テスト構造の詳細評価

- **SIG 保守性と深刻な論理バグ (`BillingManager.java`)**: `processBatchBilling` は 57 行・分岐数 13・**認知的複雑度** `56`・引数 `8` 個（SIG 上限 6 超過）の巨大メソッドとなっている。特に `status == "PAID"`（52行目）、`currency == "USD"`（55行目）、`currency == "JPY"`（57行目）の3箇所で文字列リテラルを `==` 参照比較しているため、JSON デシリアライズ等で生成された別インスタンスの文字列に対して税率計算が一切適用されない致命的バグが存在する。さらに 79 行目で `LegacyDateHelper` に未定義の `currentPeriodLabel()` を呼び出し、83 行目の空 `catch (Exception e) {}` で例外を握り潰している。

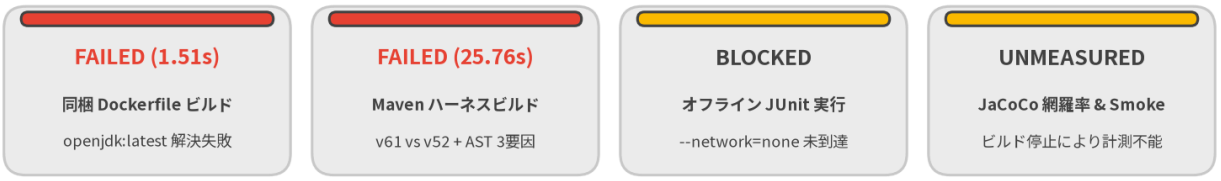
- **言語ターゲット不整合・未宣言依存・共有状態 vs 旧API負債**:
  - **AST 言語バージョン不整合 (`language_version_mismatches=1`)**: `pom.xml` が `maven.compiler.source=1.8` を宣言しているにもかかわらず、`GatewayClient.java:4-6` で **Java 11 導入の** `java.net.http.HttpClient` / `HttpRequest` / `HttpResponse` を使用している。
  - **マニフェスト未宣言依存 (`undeclared_dependencies=2`)**: `BillingManager.java:12` が `org.springframework.transaction.annotation.Transactional` を import しているが、`pom.xml` には `spring-tx`（または `spring-boot-starter-jdbc` / `data-jpa`）が宣言されていない（さらに 91 行目の `saveAuditTrail` は `private` メソッドのため AOP プロキシも無効）。
  - **スレッド間競合状態と旧 API 負債の峻別**: `BillingController.channel` と `BillingManager.INVOICE_CACHE`（`public static HashMap`）は並行リクエスト間で共有される実働の競合バグである一方、`LegacyDateHelper` の `new SimpleDateFormat(...)` と未使用 `collectAuditTags` の `Vector` はメソッドローカル生成のため直接のスレッド競合ではなく、都度生成コストと旧 `java.util.Date` 系のモダナイゼーション負債に該当する。
- **形骸化した JUnit 4 テスト**: `BillingManagerTest.java` は JUnit 4 (`org.junit.Test`) を使用し、`com.example.billing` パッケージ直下に置かれている上、`testSmokeRun` 内で `InvoiceRecord` を生成して `getAmountCents()` の戻り値を捨てるのみで **アサーションが 0 件** である。

# コンテナ動的検証（RV-01〜RV-04 ビルド失敗原文と4重のコンパイル阻害要因）

隔離 Docker 環境における動的検証（`run_check`）では、同梱 `Dockerfile` ビルド（`openjdk:latest` 解決失敗）と PNI Maven ハーネスビルドの双方が失敗し、後続の単体テスト・JaCoCo カバレッジ・起動確認がすべて `BLOCKED` となった。Maven ハーネスのコンパイラ出力原文（抜粋）は以下の通りである。

- **コンパイラ失敗ログ原文 (`harness_build`)**:
  - `bad class file: /m2repo/org/springframework/boot/spring-boot/4.2.0-RC1/spring-boot-4.2.0-RC1.jar(org/springframework/boot/SpringApplication.class)`
  - `class file has wrong version 61.0, should be 52.0`

- **解消すべき4つの独立したコンパイル阻害要因**: `javac` は最初のクラスファイル読込エラー（`version 61.0 vs 52.0`）で停止しているが、静的 AST 解析により **(1)** `LATEST` **->** `4.2.0-RC1` **のバイトコード衝突** に加え、**(2)** `GatewayClient.java` **の Java 11** `java.net.http.*` **使用、(3)** `BillingManager.java` **の** `spring-tx` **依存欠落、(4)** `dateHelper.currentPeriodLabel()` **未定義メソッド呼び出し** の計4箇所を同時に解消しなければクリーンビルドおよび後続の隔離ループバック DAST（`web_pentest`：`15.74s` エラー停止）に到達しないことが確定している。

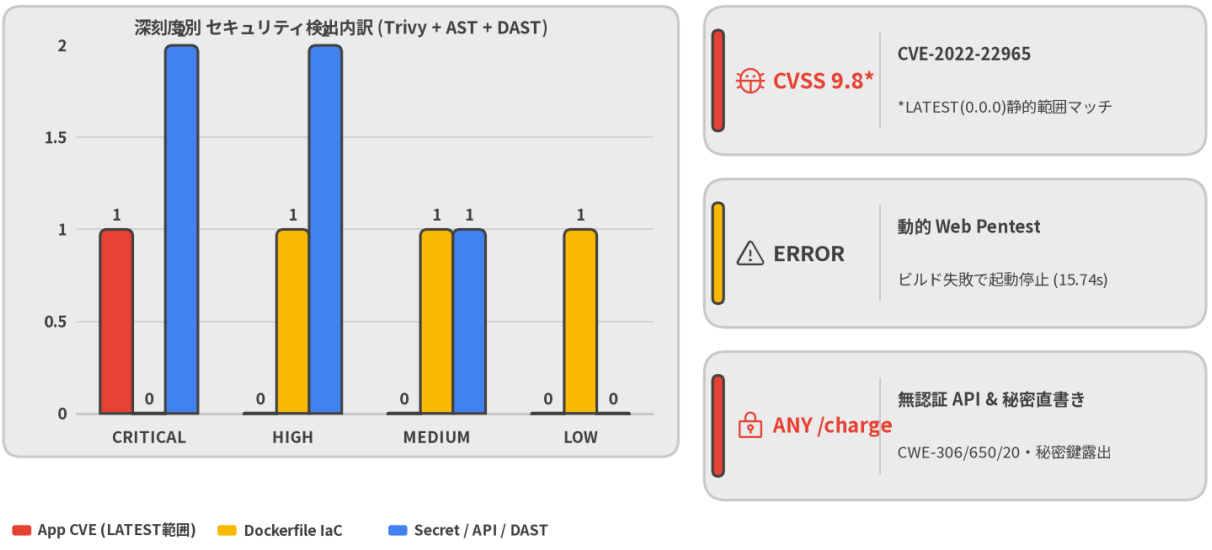



| 検証ゲート | ステータス | 所要時間 | 決定的エラー出力・原因エビデンス |
|---|---|---|---|
| RV-01a: 同梱 Dockerfile | FAILED (exit 1) | 1.51s | FROM openjdk:latest メタデータ解決失敗。加えて mvn package 工程がなく存在しない billing.jar を CMD 指定しているため起動不能 |
| RV-01b: Maven クリーンビルド | FAILED (exit 1) | 25.76s | bad class file: spring-boot-4.2.0-RC1.jar (wrong version 61.0, should be 52.0)。他に Java 11 HttpClient・spring-tx欠落・未定義呼出も潜在 |
| RV-02: オフライン単体テスト | BLOCKED | 0.00s | ハーネスビルド失敗によりブロック。加えて静的解析上も JUnit 4 テスト1件・アサーション 0件であり実質的な検証能力を欠く |
| RV-03: JaCoCo 実測カバレッジ | UNMEASURED (measured=false) | 0.00s | jacoco-maven-plugin:0.8.12 準備段階でコンパイル停止したため行・分岐網羅率は未計測 (静的テスト対応率も 33.3% に留まる) |
| RV-04: Web API スモーク起動 | BLOCKED (web_api) | 0.00s | ビルド失敗により ApplicationContext 起動未到達 (SEC-06 web_pentest も同様のビルドエラーで停止)。循環DIエラーリスクも併存 |

コンテナ動的検証KPIと RV-01〜RV-04 ゲート別失敗・ブロック内訳

# 6. 詳細分析：セキュリティ・脆弱性・動的DAST検証

本章では、Trivy ( `0.69.3` ) によるファイルシステム SCA（ソフトウェア構成解析）・ `Dockerfile` IaC 設定検査、AST によるハードコード秘密情報・CWE 危険シンク・HTTP API エンドポイント境界・GitHub Actions サプライチェーン検査、および隔離ループバック環境による動的 Web ペネトレーション監査（ `SEC-06 / web_pentest` ）の結果を統合して評価する。




Trivy アプリケーションCVE・Dockerfile設定不備・API/DASTリスク内訳

## セキュリティゲート（SEC-01〜SEC-05）および CI サプライチェーンの詳細評価

- **SEC-01（非固定 `LATEST` 指定の静的範囲マッチ vs Maven 動的解決の整合性）**: `pom.xml` の `org.springframework.boot:spring-boot-starter-web@LATEST` （ `is_unpinned_specifier=true` ）に対し、Trivy の静的 `pom.xml` アナライザはシンボル指定 `LATEST` を下限値 `0.0.0` と解釈するため、影響範囲 `< 2.6.6` の `CVE-2022-22965` **（Spring4Shell, CVSS 9.8）に範囲マッチ** した。一方、第5章の Maven 動的ビルド時には

中央リポジトリ上の最新 `4.2.0-RC1`（`CVE-2022-22965` 自体は修正済みだが Java 17+ 必須）が解決されて Java 1.8 と衝突している。すなわち両事象は矛盾ではなく、**「ロック・BOM 固定を欠く `LATEST` 指定が、静的監査上の脆弱範囲判定と実行時の破壊的バージョン流入を同時に招いている供給網欠陥」** という単一の根本原因に帰着する。なおコンテナ OS 層スキャン（`os_vuln_counts`）はイメージビルド失敗により未走査である。

- **SEC-02（Dockerfile ハードニング）**: Trivy IaC スキャンにより `Dockerfile` から `DS-0002`（`HIGH`：非 root `USER` 未指定による root 実行）、`DS-0001`（`MEDIUM`：`openjdk:latest` タグ使用）、`DS-0026`（`LOW`：`HEALTHCHECK` 未定義）の計3件が検出された（加えて `.dockerignore` も欠落）。
- **SEC-03〜SEC-05（秘密情報・CWE シンク・無認証 `/charge` 境界）**: SQLi/CMDi 等の危険コードシンク（`SEC-04`）は `0` 件だが、`GatewayClient.java:13` に決済 API のライブ秘密鍵定数（`GATEWAY_SECRET = "rk_live_***[REDACTED]***"`）が直書きされている（`SEC-03`）。さらに `BillingController.java:22-27` の `@RequestMapping("/charge")` は全 HTTP メソッド（`ANY`）を受け付け、認証・認可機構を一切持たず、未検証の生 `Map` ペイロードに対して `Long.parseLong` を直接実行している（`SEC-05`）。
- **CI サプライチェーン（OpenSSF Scorecard 観点）**: `.github/workflows/build.yml` は `pull_request_target` トリガー下で `actions/checkout@main`（SHA 非固定）と `mvn package` を実行し、`permissions:` 制限も欠く極めて危険な構成となっている。

| ゲート / 深刻度 | 識別子・ルール | 対象箇所 | 検出内容・攻撃面 | 必須緩和・修正策 |
| --- | --- | --- | --- | --- |
| SEC-01 [CRITICAL*] | CVE-2022-22965 (LATEST範囲一致) | pom.xml:20 spring-boot-web | LATESTを0.0.0基準で範囲照合 (Maven実行時は4.2-RC1が流入) | LATEST廃止・Spring 3.4固定BOM化後にOS層含め再スキャン |
| SEC-03 [CRITICAL] | Hardcoded Secret (OWASP ASVS) | GatewayClient .java:13 | GATEWAY_SECRET に本番形式鍵 (rk_live_[REDACTED]) 直書き | 定数削除・鍵失効ローテーション・Secret Manager 注入化 |
| CI 供給網 [CRITICAL] | OpenSSF Token / Pinned-Deps | .github/workflows/ build.yml:4 | pull_request_target + 権限無制限 + actions/checkout@main | pull_request 化・contents:read 明記・フルコミットSHA固定 |
| SEC-05 [HIGH] | Unauthenticated API Boundary | BillingController .java:22-27 | ANY /charge が無認証・HTTP動詞無制限・parseLong無検証で公開 | @PostMapping 限定・Spring Security認証・@Valid DTO化 |
| SEC-02 [HIGH] | Trivy DS-0002 | Dockerfile | USER 命令が存在せず最終コンテナプロセスが root 権限で動作 | 専用グループ・非 root USER を作成し実行権限を最小化 |
| SEC-02 [MEDIUM] | Trivy DS-0001 | Dockerfile:1 | FROM openjdk:latest の可変・廃止タグ指定 (.dockerignore不在) | eclipse-temurin:21-jre 等の固定タグ/ダイジェストを採用 |
| SEC-02 [LOW] | Trivy DS-0026 | Dockerfile | コンテナ HEALTHCHECK 命令が定義されていない | Actuator /actuator/health を用いた HEALTHCHECK を定義 |
| SEC-04 [PASS] | CWE-78 / 89 / 94 / 502 SAST | src/main/java 全6ファイル | SQLi・コマンド実行・危険な逆直列化シンクは 0 件 | 今後のDB導入時もパラメータ化クエリ方針を維持 |

無認証 ANY /charge エンドポイント・ハードコード秘密情報・LATEST範囲CVE・IaC 詳細一覧

# 動的 Web ペネトレーション監査（SEC-06：`web_pentest` 起動ブロック診断と `.pni/pentests/` スイート構成）

新しく追加された動的 Web ペネトレーション監査ステップ（`act-15: web_pentest`）は、`.pni/pentests/examples/java/api_spring_messy/` 配下の 4 ファイル（`conftest.py`, `test_app_probes.py`, `test_baseline_http.py`, `test_target_surface.py`）を検証スイートとしてロードし、Docker コンテナによる隔離ループバック起動を試行した。しかし Maven ビルド不整合によりコンテナ構築段階で失敗し、ライブ HTTP プローブ実行は `status="error"`（`server_booted=false`, `duration_sec=15.74`）で停止した。

- **起動失敗ログ原文 (`PentestFacts.boot_failure_reason`)**:
  - `Container build failed: ERROR: failed to build: failed to solve: process "/bin/sh -c mvn -B -Dmaven.repo.local=/m2repo org.jacoco:jacoco-maven-`

`plugin:${JACOCO_VERSION}:prepare-agent test -DfailIfNoTests=false -Dmaven.test.failure.ignore=tru`

- **整備済み DAST プローブとコード欠陥の対応関係**: ライブ通信自体はビルド障害でブロックされたものの、`test_app_probes.py` およびベースラインスイートが検査対象とする 4 種類の脆弱性プローブは、`BillingController.chargeCustomer`（`BillingController.java:22-27`）の静的欠陥と 1 対 1 で一致している。ビルド復旧直後にこれらのプローブが検出し、移行フェーズの完了ゲートとして機能する検証項目を以下に整理する。




| プローブファイル / テスト名 | 対象CWE / 重要度 | 対象コード箇所 | 検証内容と現状の内在脆弱性 (ビルド復旧後の期待防御) |
|---|---|---|---|
| test_app_probes.py requires_authentication | CWE-306 [HIGH] | BillingController .java:22 | 無認証 POST /charge 送信に対し 401/403 を要求。現状は認証機構が一切なく 200 OK で課金処理が実行されるため Spring Security 必須 |
| test_app_probes.py rejects_get_verb... | CWE-650 [MEDIUM] | BillingController .java:22 | PUT /charge 送信に対し 405 Method Not Allowed を要求。現状は @RequestMapping("/charge") が全HTTP動詞を受理するため @PostMapping 化 |
| test_app_probes.py handles_malformed_amount | CWE-20 [MEDIUM] | BillingController .java:27 | amountCents="not-a-number" に対し 400/422 を要求。現状は未検証の Long.parseLong が NumberFormatException (HTTP 500) を誘発 |
| test_baseline_http.py & test_target_surface.py | CWE-693 / 538 / 942 / 22 [LOW-MED] | Spring Security / HTTP 境界全体 | X-Content-Type-Options: nosniff、CORS ワイルドカード抑止、.env/.git 非露出、SQLi/XSS/パストラバーサル耐性のベースライン自動検証 |

SEC-06 動的Webペネトレーション監査 (web_pentest) 実行ステータスと .pni/pentests プローブ構成